

Distributional Reinforcement Learning with Function Approximation

Abstract

These notes give an overview of some theoretical foundations behind approximation based distributional reinforcement learning and newer methods using differentiable metrics on probability measures. We begin with linear value-function approximation in the standard scalar case, then we replace scalar values by return distributions and introduce the two classic representations, quantile and categorical, of probability distributions. Then, we go over how these representations can be used in function approximation using deep neural networks. Finally, we cover the recent research on distributional reinforcement learning using differentiable metrics on probability measures such as the MMD distances or Sinkhorn divergence.

Preliminaries

The first part of these notes follow the treatment of distributional reinforcement learning in [BDR23]. We first recall some of the preliminary definitions and results.

Definition 1: Finite Markov decision process and policy

A finite Markov decision process is given by

$$(\mathcal{X}, \mathcal{A}, \xi_0, P_X, P_R),$$

where $\mathcal{X}$ is a finite state space, $\mathcal{A}$ is a finite action space, ξ_0 is an initial-state distribution, $P_X(\cdot \mid x, a)$ is the next-state kernel, and $P_R(\cdot \mid x, a)$ is the reward distribution. A policy is a map

$$\pi : \mathcal{X} \rightarrow \mathcal{P}(\mathcal{A}), \quad x \mapsto \pi(\cdot \mid x),$$

and the discount factor satisfies $\gamma \in [0, 1)$. The markov chain induced by P_X and π gives rises to the trajectory measure P_π .

Given a policy π , the trajectory process is

$$\begin{aligned} X_0 &\sim \xi_0, & A_t &\sim \pi(\cdot \mid X_t), \\ R_t &\sim P_R(\cdot \mid X_t, A_t), & X_{t+1} &\sim P_X(\cdot \mid X_t, A_t), & t \geq 0. \end{aligned}$$

Starting from $X_0 = x$ and then following π , define the discounted return

$$G^\pi(x) = \sum_{t=0}^{\infty} \gamma^t R_t.$$

Definition 2: Value function and return distribution

The state-value function is the expected return

$$V^\pi(x) = \mathbb{E}_\pi[G^\pi(x)].$$

The return distribution at state x is

$$\eta_\pi(x) = \text{Law}(G^\pi(x)).$$

Thus the scalar value function is the first moment of the return distribution:

$$V^\pi(x) = \int_{\mathbb{R}} z \eta_\pi(x)(dz).$$

Definition 3: Bellman operator for scalar values

For a function $V : \mathcal{X} \rightarrow \mathbb{R}$, the Bellman operator for policy π is

$$(\mathcal{T}^\pi V)(x) = \mathbb{E}_{A \sim \pi(\cdot | x)} [\mathbb{E} [R + \gamma V(X') \mid X = x, A]].$$

Equivalently, the outer expectation samples the action from the policy and the inner expectation samples reward and next state from the MDP. The Bellman equation is

$$V^\pi = \mathcal{T}^\pi V^\pi.$$

The scalar Bellman equation comes from a stronger identity at the level of random variables. With

$$A \sim \pi(\cdot \mid x), \quad R \sim P_R(\cdot \mid x, A), \quad X' \sim P_X(\cdot \mid x, A),$$

the return satisfies

$$G^\pi(x) \stackrel{D}{=} R + \gamma G^\pi(X') \quad \text{given } X = x.$$

Taking laws gives

$$\eta_\pi(x) = \text{Law}(R + \gamma G^\pi(X') \mid X = x).$$

Definition 4: Distributional Bellman operator

For $r, z \in \mathbb{R}$, define the bootstrap map

$$b_{r,\gamma}(z) = r + \gamma z.$$

If $\nu \in \mathcal{P}(\mathbb{R})$ and $Z \sim \nu$, then the pushforward $(b_{r,\gamma})_{\#}\nu$ is the law of $r + \gamma Z$:

$$(b_{r,\gamma})_{\#}\nu = \text{Law}(r + \gamma Z).$$

For a return-distribution function $\eta : \mathcal{X} \rightarrow \mathcal{P}(\mathbb{R})$, the distributional Bellman operator is

$$(\mathcal{T}^\pi \eta)(x) = \mathbb{E}_{A \sim \pi(\cdot|x)} [\mathbb{E} [(b_{R,\gamma})_{\#}\eta(X') \mid X = x, A]].$$

The return-distribution function η_π is its fixed point:

$$\eta_\pi = \mathcal{T}^\pi \eta_\pi.$$

Definition 5: Finite representations of return distributions

For a probability measure $\eta \in \mathcal{P}(\mathbb{R})$, let

$$F_\eta(t) = \eta((-\infty, t]), \quad F_\eta^{-1}(\tau) = \inf \{z \in \mathbb{R} : F_\eta(z) \geq \tau\}.$$

Two standard finite representations are:

$$\text{categorical:} \quad \eta(x) = \sum_{i=1}^m p_i(x) \delta_{\theta_i}, \quad p_i(x) \geq 0, \quad \sum_{i=1}^m p_i(x) = 1,$$

and

$$\text{quantile:} \quad \eta(x) = \frac{1}{m} \sum_{i=1}^m \delta_{\theta_i(x)}, \quad \tau_i = \frac{2i-1}{2m}, \quad \theta_i(x) \approx F_{\eta(x)}^{-1}(\tau_i).$$

If $\mathcal{X}$ is large, storing a scalar value for every state is already expensive; storing a full probability distribution for every state is even more so. The rest of the notes is about approximation: first for scalar values, and then for return distributions.

1 Linear Function Approximation

We first discuss linear approximation for the state-value function

$$V^\pi : \mathcal{X} \rightarrow \mathbb{R}.$$

This is the cleanest setting because the input is only the state. This section follows Chapter 9 of [BDR23].

1.1 Function Approximation and Aliasing

In a small state space, V^π can be stored as a table with one number per state. Function approximation instead shares parameters across states. This is necessary when $\mathcal{X}$ cannot be enumerated and it permits generalization to states not yet observed. The cost is *aliasing*: a shared parameter changes the prediction at several states. If two aliased states require different values, no parameter vector represents both exactly. With bootstrapping, the resulting error may propagate to states that are not themselves aliased.

Definition 6: State representation and linear value approximation

Let $n \in \mathbb{N}$. A state representation is a mapping

$$\phi : \mathcal{X} \rightarrow \mathbb{R}^n, \quad \phi(x) = \begin{pmatrix} \phi_1(x) \\ \vdots \\ \phi_n(x) \end{pmatrix}.$$

For $w \in \mathbb{R}^n$, the linear value function approximation is

$$V_w(x) = \phi(x)^\top w,$$

where each ϕ_i is a basis function and $\phi_i(x)$ is a feature.

The model is linear in its weights and

$$V_{\alpha w_1 + \beta w_2} = \alpha V_{w_1} + \beta V_{w_2}, \quad \nabla_w V_w(x) = \phi(x).$$

For finite $\mathcal{X}$, collect the row vectors $\phi(x)^\top$ in a feature matrix $\Phi \in \mathbb{R}^{|\mathcal{X}| \times n}$. Then

$$V_w = \Phi w, \quad \mathcal{F}_\phi = \{\Phi w : w \in \mathbb{R}^n\}.$$

For action values, either take $\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^n$ and $Q_w(x, a) = \phi(x, a)^\top w$, or use a common state representation with one weight vector per action, $Q_w(x, a) = \phi(x)^\top w_a$.

1.2 Optimal Approximation in a Weighted Norm

Let $\xi \in \mathcal{P}(\mathcal{X})$ express the relative importance of the states. Define

$$\|V - V'\|_{\xi, 2} = \left(\sum_{x \in \mathcal{X}} \xi(x) (V(x) - V'(x))^2 \right)^{1/2}.$$

The best approximation to V^π solves

$$\min_{w \in \mathbb{R}^n} \|V^\pi - V_w\|_{\xi, 2}.$$

Let $\Xi = \text{diag}(\xi(x) : x \in \mathcal{X})$. If ξ has full support and the columns of Φ are linearly independent, the unique solution is

$$w^* = (\Phi^\top \Xi \Phi)^{-1} \Phi^\top \Xi V^\pi.$$

Thus the state weighting ξ is part of the approximation problem: changing how often states are sampled generally changes the best attainable weights.

1.3 The Projected Bellman Operator

In general, $V \in \mathcal{F}_\phi$ does not imply $\mathcal{T}^\pi V \in \mathcal{F}_\phi$. Define the weighted orthogonal projection $\Pi_{\phi,\xi}$ by

$$(\Pi_{\phi,\xi} V)(x) = \phi(x)^\top w^*, \quad w^* \in \arg \min_{w \in \mathbb{R}^n} \|V - V_w\|_{\xi,2}.$$

Approximate dynamic programming iterates

$$V_{k+1} = \Pi_{\phi,\xi} \mathcal{T}^\pi V_k.$$

Projection removes the component of the Bellman update that the features cannot represent. Repeated projection can compound approximation error.

Let ξ_π be a fully supported steady-state distribution under π . Then P^π and Π_{ϕ,ξ_π} are nonexpansions in the ξ_π -weighted L_2 norm. Consequently,

$$\|\Pi_{\phi,\xi_\pi} \mathcal{T}^\pi V - \Pi_{\phi,\xi_\pi} \mathcal{T}^\pi V'\|_{\xi_\pi,2} \leq \gamma \|V - V'\|_{\xi_\pi,2}.$$

The projected operator has a unique fixed point

$$\hat{V}^\pi = \Pi_{\phi,\xi_\pi} \mathcal{T}^\pi \hat{V}^\pi,$$

and the book's sharper orthogonal-projection argument gives

$$\|\hat{V}^\pi - V^\pi\|_{\xi_\pi,2} \leq \frac{1}{\sqrt{1-\gamma^2}} \|\Pi_{\phi,\xi_\pi} V^\pi - V^\pi\|_{\xi_\pi,2}.$$

The fixed point $\hat{V}^\pi$ is generally not the direct best approximation $\Pi_{\phi,\xi_\pi} V^\pi$.

1.4 Monte Carlo and Semi-Gradient TD

For independent state–return samples (x_k, g_k) , stochastic gradient descent on $(g_k - \phi(x_k)^\top w)^2$ gives

$$w \leftarrow w + \alpha_k (g_k - \phi(x_k)^\top w) \phi(x_k).$$

For a sampled transition (x_k, a_k, r_k, x'_k) , replace g_k by a bootstrapped target. The TD error is

$$\delta_k = r_k + \gamma V_w(x'_k) - V_w(x_k) = r_k + \gamma \phi(x'_k)^\top w - \phi(x_k)^\top w.$$

The semi-gradient update differentiates only the current prediction $V_w(x_k)$, treating the bootstrap target as fixed:

$$w \leftarrow w + \alpha_k \delta_k \phi(x_k).$$

Equivalently, if a fixed target vector $\tilde{w}$ is used, the update is SGD on

$$\frac{1}{2} \left(r_k + \gamma \phi(x'_k)^\top \tilde{w} - \phi(x_k)^\top w \right)^2.$$

At a projected Bellman fixed point $\widehat{V}^\pi = \Phi \widehat{w}$, the expected TD error is orthogonal to all features:

$$\mathbb{E}_\pi \left[\left(R + \gamma \phi(X')^\top \widehat{w} - \phi(X)^\top \widehat{w} \right) \phi(X) \right] = 0.$$

This is the sample-based form of the projected Bellman equation. The representation ϕ is therefore not a cosmetic choice: it determines which Bellman fixed points are expressible.

The same idea works for action-values

$$Q^\pi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}.$$

Now the feature map takes a state-action pair,

$$\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^d,$$

and the approximation is

$$Q_w(x, a) = \phi(x, a)^\top w.$$

The TD error for policy evaluation becomes

$$\delta_t = r_t + \gamma Q_{w_t}(x'_t, a'_t) - Q_{w_t}(x_t, a_t), \quad a'_t \sim \pi(\cdot \mid x'_t),$$

with update

$$w_{t+1} = w_t + \alpha_t \delta_t \phi(x_t, a_t).$$

We will use this action-value version below because QR-DQN and the deep distributional algorithms are control methods: the agent must compare actions in the same state, and this comparison is immediate from $Q(x, a)$.

2 Distributional Value Approximation

Distributional reinforcement learning replaces the scalar value function by a return-distribution function. Instead of storing only the mean $Q^\pi(x, a) = \mathbb{E}[G^\pi(x, a)]$, we keep the law of the return:

$$\eta^\pi(x, a) = \text{Law}(G^\pi(x, a)).$$

Thus

$$\eta^\pi : \mathcal{X} \times \mathcal{A} \rightarrow \mathcal{P}(\mathbb{R}),$$

where $\mathcal{P}(\mathbb{R})$ denotes probability distributions on the real line. The classical action-value function is recovered by taking expectations:

$$Q^\pi(x, a) = \int_{\mathbb{R}} z \eta^\pi(x, a)(dz).$$

The distributional Bellman operator is most cleanly written using pushforwards. For $r \in \mathbb{R}$, define the affine map

$$f_{r,\gamma}(z) = r + \gamma z.$$

If $\nu \in \mathcal{P}(\mathbb{R})$, then $(f_{r,\gamma})_\# \nu$ is the distribution of $r + \gamma Z$ when $Z \sim \nu$. The distributional Bellman operator $\mathcal{T}^\pi$ maps a return-distribution function η to

$$(\mathcal{T}^\pi \eta)(x, a) = \mathbb{E}[(f_{R,\gamma})_\# \eta(X', A') \mid X = x, A = a], \quad A' \sim \pi(\cdot \mid X').$$

The expectation averages over the random reward, the next state, and the next action sampled from the policy. The distributional Bellman equation is

$$\eta^\pi = \mathcal{T}^\pi \eta^\pi.$$

Taking means on both sides recovers the usual Bellman equation for Q^π , but the distributional equation is stronger: it describes the whole random return, not only its expectation. Just as linear TD restricts Q to a class $\mathcal{Q}_\phi$, distributional methods restrict η to a class of approximate distributions. The quantile family is one of the most important such choices. For a distribution ν with cumulative distribution function F_ν , define its quantile function by

$$F_\nu^{-1}(\tau) = \inf\{z \in \mathbb{R} : F_\nu(z) \geq \tau\}, \quad \tau \in (0, 1).$$

Choose N quantile levels, usually the midpoints

$$\tau_i = \frac{2i-1}{2N}, \quad i = 1, \dots, N.$$

An N -quantile approximation represents $\eta(x, a)$ by the empirical distribution

$$\eta_\theta(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(x, a)},$$

where δ_z is a Dirac mass at z . The parameter $\theta_i(x, a)$ is interpreted as the predicted τ_i -quantile of the return distribution at (x, a) .

The direct distributional analogue of linear action-value approximation is to make each quantile location linear in the same feature vector:

$$\theta_i(x, a) = \phi(x, a)^\top w_i, \quad w_i \in \mathbb{R}^d.$$

Equivalently, with $W = (w_1, \dots, w_N) \in \mathbb{R}^{d \times N}$,

$$\eta_W(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\phi(x, a)^\top w_i}.$$

The scalar approximation $Q_w(x, a)$ has become a measure-valued approximation $\eta_W(x, a)$. The same features are now asked to explain not only the average return, but also the spread, asymmetry, and tail behavior of the return.

3 Quantile TD

Quantile TD applies temporal-difference learning to the quantile parameterization. For policy evaluation, sample a transition and next action

$$(x_t, a_t, r_t, x'_t, a'_t), \quad a'_t \sim \pi(\cdot \mid x'_t).$$

The distributional Bellman target is represented by the N target atoms

$$y_j = r_t + \gamma \theta_j(x'_t, a'_t), \quad j = 1, \dots, N.$$

The current prediction at (x_t, a_t) has atoms

$$z_i = \theta_i(x_t, a_t), \quad i = 1, \dots, N.$$

Quantile regression uses the asymmetric absolute loss

$$\rho_\tau(u) = u(\tau - \mathbb{1}_{\{u < 0\}}).$$

The empirical Quantile TD loss is then

$$L(W) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}(y_j - z_i).$$

The residual $y_j - z_i$ compares an atom of the Bellman target distribution with the predicted τ_i -quantile at the current state-action pair. Minimizing this loss moves $\eta_W(x_t, a_t)$ toward

$$\frac{1}{N} \sum_{j=1}^N \delta_{r_t + \gamma \theta_j(x'_t, a'_t)} = (f_{r_t, \gamma})_{\#} \eta_W(x'_t, a'_t).$$

For linear quantiles, a subgradient update for each head is

$$w_{i,t+1} = w_{i,t} + \alpha_t \left(\frac{1}{N} \sum_{j=1}^N (\tau_i - \mathbb{1}_{\{y_j < z_i\}}) \right) \phi(x_t, a_t), \quad i = 1, \dots, N.$$

This is the exact analogue of the semi-gradient TD update:

$$\delta_t = r_t + \gamma Q_{w_t}(x'_t, a'_t) - Q_{w_t}(x_t, a_t) \rightsquigarrow \{y_j - z_i\}_{i,j=1}^N.$$

The scalar TD target is replaced by a Bellman target distribution, and the single scalar TD error is replaced by all pairwise quantile residuals.

In implementations, one often replaces ρ_τ by a quantile Huber loss. This is the smooth loss used in QR-DQN; see the treatment in [BDR23, Chapter 10]. Define

$$\mathcal{L}_\kappa(u) = \begin{cases} \frac{1}{2}u^2, & |u| \leq \kappa, \\ \kappa(|u| - \frac{1}{2}\kappa), & |u| > \kappa, \end{cases}$$

and

$$\rho_\tau^\kappa(u) = |\tau - \mathbb{1}_{\{u < 0\}}| \frac{\mathcal{L}_\kappa(u)}{\kappa}.$$

The corresponding loss is

$$L_\kappa(W) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}^\kappa(y_j - z_i).$$

The Huber version keeps the asymmetric quantile penalty but smooths the non-differentiability around zero, which is useful when the quantile locations are produced by a neural network. For control, the next action is usually selected greedily with respect to the mean of the predicted distribution:

$$a^*(x') \in \arg \max_{a' \in \mathcal{A}} \frac{1}{N} \sum_{j=1}^N \theta_j(x', a').$$

The target atoms become

$$y_j = r + \gamma \theta_j(x', a^*(x')), \quad j = 1, \dots, N.$$

This is the quantile analogue of replacing policy evaluation TD by Q-learning.

4 Gateway to Deep Representations

The linear approximation above separates representation and prediction:

$$\text{fixed features } \phi(x, a) \quad \text{and} \quad \text{linear heads } w_i.$$

This is the useful gateway into deep distributional reinforcement learning. The deep version keeps the same conceptual decomposition, but replaces the fixed feature map by a learned representation. A neural network maps the observation or state to a feature vector

$$\psi_\xi(x) \in \mathbb{R}^d,$$

and the final layers produce a distributional prediction. In the quantile case one may write

$$\theta_i(x, a; \omega) = f_i(\psi_\xi(x), a), \quad i = 1, \dots, N.$$

The return-distribution approximation becomes

$$\eta_\omega(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(x, a; \omega)}.$$

The parameter ω includes both the representation parameters ξ and the prediction heads. Linear Quantile TD is recovered as the special case in which ψ_ξ is fixed and f_i is linear. Deep quantile methods instead optimize all of ω by backpropagating through the distributional TD loss.

5 QR-DQN

QR-DQN is the deep control version of Quantile TD. It uses a neural network to output N quantile locations for each action:

$$\theta_i(x, a; \omega), \quad i = 1, \dots, N, \quad a \in \mathcal{A}.$$

For each state-action pair, the predicted return distribution is

$$\eta_\omega(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(x, a; \omega)}.$$

The predicted scalar action-value is the mean of this empirical distribution:

$$Q_\omega(x, a) = \frac{1}{N} \sum_{i=1}^N \theta_i(x, a; \omega).$$

Thus QR-DQN can still choose greedy actions in the usual value-based way, but the value used for action selection is obtained by averaging the learned quantiles.

As in DQN, QR-DQN uses two sets of parameters. The online network has parameters ω and is updated by gradient descent. The target network has parameters $\bar{\omega}$ and is held fixed for several updates before being copied from ω . For a transition

$$(x, a, r, x', d),$$

where $d \in \{0, 1\}$ indicates termination, the greedy next action is selected by the mean of the target-network distribution:

$$a^*(x') \in \arg \max_{a' \in \mathcal{A}} \frac{1}{N} \sum_{j=1}^N \theta_j(x', a'; \bar{\omega}).$$

The target quantile atoms are

$$y_j = r + \gamma(1 - d)\theta_j(x', a^*(x'); \bar{\omega}), \quad j = 1, \dots, N.$$

The online prediction atoms for the sampled action are

$$z_i = \theta_i(x, a; \omega), \quad i = 1, \dots, N.$$

QR-DQN minimizes the pairwise quantile Huber loss

$$L_{\text{QR}}(\omega) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}^{\kappa}(y_j - z_i),$$

with quantile levels

$$\tau_i = \frac{2i - 1}{2N}.$$

In a minibatch $\mathcal{B}$ sampled from replay memory, the objective is

$$\mathcal{L}_{\text{QR}}(\omega) = \frac{1}{|\mathcal{B}|} \sum_{(x, a, r, x', d) \in \mathcal{B}} L_{\text{QR}}(\omega; x, a, r, x', d).$$

The target atoms are treated as constants when differentiating with respect to ω ; this is the usual semi-gradient idea inherited from TD learning and DQN.

The quantile Huber loss is important because the raw quantile loss $\rho_{\tau}(u)$ is non-smooth at zero. QR-DQN uses

$$\mathcal{L}_{\kappa}(u) = \begin{cases} \frac{1}{2}u^2, & |u| \leq \kappa, \\ \kappa \left(|u| - \frac{1}{2}\kappa \right), & |u| > \kappa, \end{cases}$$

and

$$\rho_{\tau}^{\kappa}(u) = \left| \tau - \mathbb{1}_{\{u < 0\}} \right| \frac{\mathcal{L}_{\kappa}(u)}{\kappa}.$$

For small residuals, the loss has a squared-error shape, which gives smoother gradients. For large residuals, it grows linearly, which prevents very large TD errors from dominating the update. The factor $|\tau - \mathbb{1}_{\{u < 0\}}|$ keeps the asymmetry needed for quantile regression: underestimation and overestimation are penalized differently depending on the quantile level τ .

How QR-DQN Relates to DQN

DQN learns one scalar per action:

$$x \mapsto \{Q_\omega(x, a) : a \in \mathcal{A}\}.$$

QR-DQN learns a collection of quantiles per action:

$$x \mapsto \{\theta_i(x, a; \omega) : i = 1, \dots, N, a \in \mathcal{A}\}.$$

The scalar Bellman target

$$r + \gamma \max_{a'} Q_{\bar{\omega}}(x', a')$$

is replaced by the empirical target distribution

$$\frac{1}{N} \sum_{j=1}^N \delta_{r+\gamma(1-d)\theta_j(x', a^*(x'); \bar{\omega})}.$$

The squared TD loss is replaced by the pairwise quantile Huber loss. Action selection remains risk-neutral unless we deliberately change it: QR-DQN chooses actions by the mean of the predicted quantiles. The learned object, however, is much richer than the mean.

Representation Learning Signal

The representation $\psi_\xi(x)$ in QR-DQN is trained by many losses at once. For a single transition, the network sees all residuals

$$y_j - \theta_i(x, a; \omega), \quad i, j = 1, \dots, N.$$

Lower quantile heads receive large gradients when the lower tail is poorly placed; upper quantile heads receive large gradients when the upper tail is poorly placed. Thus the shared representation is shaped not only by expected return, but by the whole geometry of the target return distribution.

This gives a useful interpretation of the empirical strength of distributional methods. A scalar DQN representation only needs to make the conditional mean easy to predict. A QR-DQN representation must make many parts of the return distribution easy to predict simultaneously. The extra prediction problem can act as a richer training signal, even when the final policy is still selected by the mean.

6 Particle Distributional DQN

QR-DQN uses particles with a special interpretation: the particle $\theta_i(x, a)$ is tied to the fixed quantile level $\tau_i = (2i - 1)/(2N)$. MMD-DQN and SinkhornDRL keep the empirical-measure form

$$\eta_\omega(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{z_i(x, a; \omega)}$$

but treat the z_i as deterministic samples of the return distribution, not as named quantiles. The common template is:

1. choose a divergence or metric d between probability measures,
2. study whether the distributional Bellman operator contracts under the induced supremum metric

$$\bar{d}(\eta, \nu) = \sup_{x,a} d(\eta(x, a), \nu(x, a)),$$

3. use the empirical version of d as a differentiable DQN loss.

The last point matters algorithmically. The ordinary Wasserstein distance is mathematically natural for distributional RL, but its empirical optimization with unrestricted particles is not as directly smooth as the losses below.

6.1 MMD-DQN

Let $k : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ be a continuous positive-definite kernel and let $\mathcal{H}_k$ be its reproducing kernel Hilbert space. For a probability measure μ , define the kernel mean embedding

$$\psi_\mu = \int k(z, \cdot) \mu(dz) \in \mathcal{H}_k.$$

The maximum mean discrepancy is

$$\text{MMD}_k(\mu, \nu) = \|\psi_\mu - \psi_\nu\|_{\mathcal{H}_k}.$$

Equivalently, if $Z, Z' \sim \mu$ and $Y, Y' \sim \nu$ are independent, then

$$\text{MMD}_k^2(\mu, \nu) = \mathbb{E}k(Z, Z') + \mathbb{E}k(Y, Y') - 2\mathbb{E}k(Z, Y).$$

If the kernel is characteristic, the embedding is injective, and $\text{MMD}_k(\mu, \nu) = 0$ implies $\mu = \nu$. Thus the MMD is then a genuine metric on probability measures.

Nguyen et al. define the state-action supremum metric [NGV20]

$$\text{MMD}_{\infty,k}(\eta, \nu) = \sup_{x,a} \text{MMD}_k(\eta(x, a), \nu(x, a)).$$

The contraction theorem does not hold for arbitrary characteristic kernels. The clean theorem assumes kernels that are translation invariant and homogeneous under discount scaling in the following sense:

$$k(x + c, y + c) = k(x, y)$$

and, for $b_{\gamma,0}(z) = \gamma z$,

$$\text{MMD}_k((b_{\gamma,0})_\# \mu, (b_{\gamma,0})_\# \nu) = \gamma^{\alpha/2} \text{MMD}_k(\mu, \nu).$$

For mixtures of such kernels, the distributional Bellman operator satisfies

$$\text{MMD}_{\infty,k}(\mathcal{T}^\pi \eta, \mathcal{T}^\pi \nu) \leq \gamma^{\alpha/2} \text{MMD}_{\infty,k}(\eta, \nu).$$

Hence the metric-first story mirrors the Wasserstein contraction story: if the chosen metric contracts under the affine Bellman shift and mixing does not expand it, fixed-point arguments become available.

The important caveat is exactly the one emphasized in the presentation. The kernel used for the clean theorem is not necessarily the kernel that performs best empirically. Kernels such as

$$k_\alpha(x, y) = -\|x - y\|^\alpha$$

fit the contraction theory, whereas Gaussian kernels generally do not yield a Bellman contraction in MMD. Nevertheless, Gaussian kernels can perform better in experiments, so the theorem should be read as a stability result for a class of metrics, not as a full explanation of the best empirical kernel choice.

For a transition (x, a, r, x') and target-network parameters $\bar{\omega}$, the particle DQN target is constructed by first choosing the greedy next action

$$a^*(x') \in \arg \max_{a' \in \mathcal{A}} \frac{1}{N} \sum_{j=1}^N z_j(x', a'; \bar{\omega})$$

and then shifting the target particles:

$$y_j = r + \gamma z_j(x', a^*(x'); \bar{\omega}), \quad j = 1, \dots, N.$$

With current particles $z_i = z_i(x, a; \omega)$, the empirical MMD loss is

$$L_{\text{MMD}}(\omega) = \frac{1}{N^2} \sum_{i, i'=1}^N k(z_i, z_{i'}) + \frac{1}{N^2} \sum_{j, j'=1}^N k(y_j, y_{j'}) - \frac{2}{N^2} \sum_{i, j=1}^N k(z_i, y_j).$$

Only the terms involving current particles are differentiated with respect to ω ; the target particles are fixed by the target network.

6.2 SinkhornDRL

SinkhornDRL starts from optimal transport and replaces the hard transport problem by an entropically regularized one [Sun+22]. For a cost $c : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}_+$, define

$$\text{OT}_c(\mu, \nu) = \inf_{\pi \in \Pi(\mu, \nu)} \int c(z, y) \pi(\text{d}z, \text{d}y),$$

where $\Pi(\mu, \nu)$ is the set of couplings with marginals μ and ν . The entropic version is

$$\text{OT}_c^\varepsilon(\mu, \nu) = \inf_{\pi \in \Pi(\mu, \nu)} \left\{ \int c(z, y) \pi(\text{d}z, \text{d}y) + \varepsilon \text{KL}(\pi \mid \mu \otimes \nu) \right\}.$$

Because this regularized objective has a self-similarity bias, SinkhornDRL uses the debiased Sinkhorn divergence

$$\overline{W}_{c, \varepsilon}(\mu, \nu) = \text{OT}_c^\varepsilon(\mu, \nu) - \frac{1}{2} \text{OT}_c^\varepsilon(\mu, \mu) - \frac{1}{2} \text{OT}_c^\varepsilon(\nu, \nu).$$

For $c(x, y) = \|x - y\|^\alpha$, define the supremum divergence

$$\overline{W}_{c, \varepsilon}^\infty(\eta, \nu) = \sup_{x, a} \overline{W}_{c, \varepsilon}(\eta(x, a), \nu(x, a)).$$

Sun et al. show that the Bellman operator is contractive with a factor bounded by

$$\overline{\Delta}_\varepsilon(\gamma, \alpha) = \gamma^\alpha \left(1 - \sup_{\mu, \nu} \lambda_\varepsilon(\mu, \nu) \right) + \sup_{\mu, \nu} \lambda_\varepsilon(\mu, \nu),$$

where

$$\lambda_\varepsilon(\mu, \nu) = \frac{\varepsilon \text{KL}(\pi^* \mid \mu \otimes \nu)}{\overline{W}_{c, \varepsilon}(\mu, \nu)} \in (0, 1)$$

and π^* is an optimal regularized coupling. Thus

$$\overline{W}_{c, \varepsilon}^\infty(\mathcal{T}^\pi \eta, \mathcal{T}^\pi \nu) \leq \overline{\Delta}_\varepsilon(\gamma, \alpha) \overline{W}_{c, \varepsilon}^\infty(\eta, \nu).$$

The rate lies between the unregularized Wasserstein-type rate and the regularization-dominated rate:

$$\overline{\Delta}_\varepsilon(\gamma, \alpha) \in (\gamma^\alpha, 1).$$

The particle learning rule has the same Bellman target construction as MMD-DQN. With current empirical measure

$$\eta_\omega(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{z_i(x, a; \omega)}$$

and target empirical measure

$$\hat{\eta}_{\bar{\omega}}(x, a, r, x') = \frac{1}{N} \sum_{j=1}^N \delta_{r + \gamma z_j(x', a^*(x'); \bar{\omega})},$$

the loss is

$$L_{\text{RW}}(\omega) = \overline{W}_{c, \varepsilon}(\eta_\omega(x, a), \hat{\eta}_{\bar{\omega}}(x, a, r, x')).$$

The Sinkhorn algorithm gives a differentiable approximation to this quantity for empirical measures, which makes it usable as a neural-network loss.

Finally, the regularization parameter interpolates between two geometries:

$$\varepsilon \rightarrow 0 \quad \Rightarrow \quad \overline{W}_{c, \varepsilon}(\mu, \nu) \rightarrow 2W_c^\alpha(\mu, \nu),$$

whereas

$$\varepsilon \rightarrow +\infty \quad \Rightarrow \quad \overline{W}_{c, \varepsilon}(\mu, \nu) \rightarrow \text{MMD}_{k_\alpha}^2(\mu, \nu), \quad k_\alpha = -c.$$

This is why the presentation treats SinkhornDRL as a bridge between Wasserstein-style distributional RL and MMD-DQN: it keeps a transport-based objective while making the empirical particle loss smooth enough for deep learning.

References

- [BDR23] Marc G. Bellemare, Will Dabney, and Mark Rowland. *Distributional Reinforcement Learning*. MIT Press, 2023.
- [NGV20] Tung D. Nguyen, Sunil Gupta, and Svetha Venkatesh. “Distributional Reinforcement Learning via Moment Matching”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2020.

- [Sun+22] Jian Sun et al. “Distributional Reinforcement Learning with Regularized Wasserstein Loss”. In: *Advances in Neural Information Processing Systems*. 2022.